DAY-16 Istio Architecture Deep Dive

- Istio control plane overview
- Data plane & Envoy proxy
- Sidecar injection (manual vs automatic)
- Istio components (istiod)
- Traffic flow inside a mesh

Outcome: Clear understanding of how Istio works internally

Module 3: Installing & Setting Up Istio

- Istio installation methods
  - o istioctl
  - o Helm
- Namespace labelling
- Verifying Istio installation
- Enabling sidecar injection

Hands-on:

- Install Istio on Kubernetes
- Deploy sample application

Outcome: Learners can set up Istio independently

Mini Project 1, Mini Project 2, Mini Project 3,

Capstone project - 2

MCQs: test

* Quick Revision

- Certifications

Istio Control Plane Overview:

The control plane is responsible for managing, configuring, and securing the entire service mesh.

The main control plane component is istiod

* What is Control Plane?

The Control Plane is the brain of Istio

It does NOT handle actual traffic, but it:

Decides how traffic should flow

Enforces security policies

Distributes configuration to proxies

Think of it like:

Air Traffic Control Tower (control plane)

vs

Airplanes flying passengers (data plane)

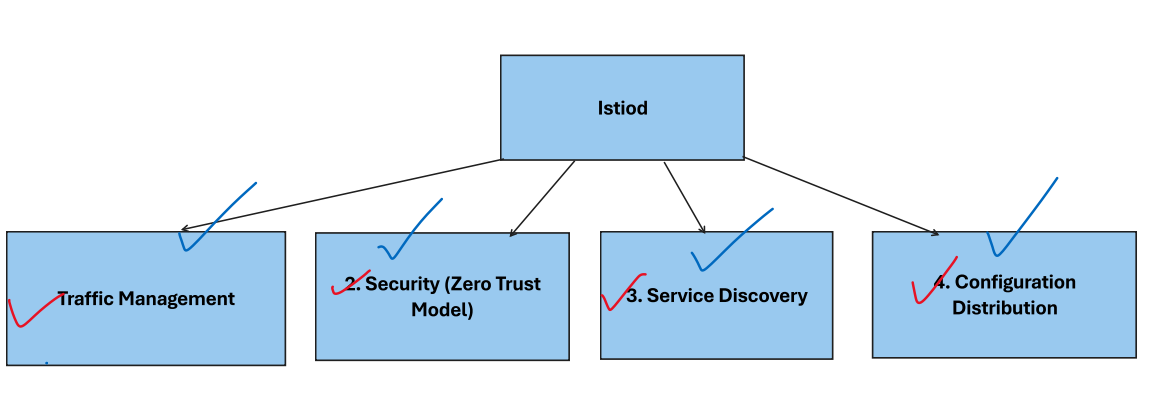
Core Component: istiod

In modern Istio, everything is consolidated into:

istiod = Single Control Plane Component

What istiod Actually Does?

Break it into 4 powerful responsibilities



1. Traffic Management:

Controls how requests move inside the mesh

Uses Istio CRDs:

[CRD - Custom Resource Definitions : CRD helps to configure traffic management, security, and observability inside Kubernetes.]

Common Istio CRDs:

VirtualService → controls routing of traffic (e.g., canary releases, A/B testing)

DestinationRule → defines policies for traffic to a service (load balancing, subsets)

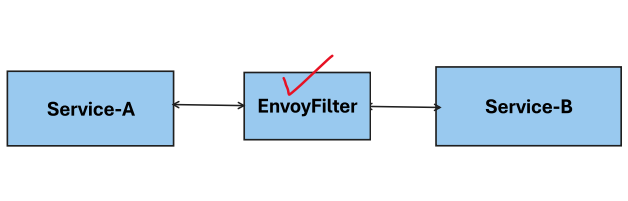
Gateway → manages inbound/outbound traffic at the edge of the mesh

ServiceEntry → allows external services to be added into the mesh

PeerAuthentication → controls mTLS settings

AuthorizationPolicy → defines access control rules

EnvoyFilter → advanced customization of the proxy behavior



Tools / Platforms that Use Istio CRDs:

Several platforms build on top of Istio and rely on its CRDs:

- Kiali – visualizes and interacts with Istio CRDs
- Kubeflow – uses Istio CRDs for routing and auth in ML pipelines
- Knative – can integrate with Istio for networking (earlier heavily dependent on it)
- OpenShift Service Mesh – wraps Istio and exposes CRDs
- Flagger – uses Istio CRDs for canary deployments

Key Capabilities:

1. Routing:

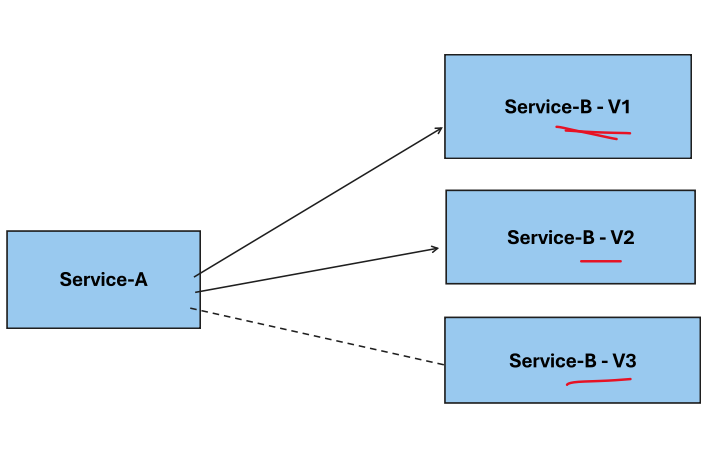
Fine-grained routing rules based on:

- URL paths

- headers

- weights

Example:
Route traffic from Service A to different versions of Service B



2. Load Balancing:

Using DestinationRule, you can choose:

- Round Robin

- Least Connections

- Random

3. Canary Deployments

Gradually shift traffic between versions:

v1 → 80%

v2 → 20%

4. Fault Injection:

Simulate failures to test resilience:

Add delays
Return errors

Example Flow:

```
User → Service A → Service B
                  ├─ v1 (80%)
                  └─ v2 (20%)
```

How It Works Together

- VirtualService → says “send 80% to v1, 20% to v2”

- DestinationRule → defines what v1 and v2 actually are (subsets)

- Gateway → exposes this routing to external users (if needed)

2. Security (Zero Trust Model)

Instead of trusting services based on network location (IP, subnet), Istio enforces:

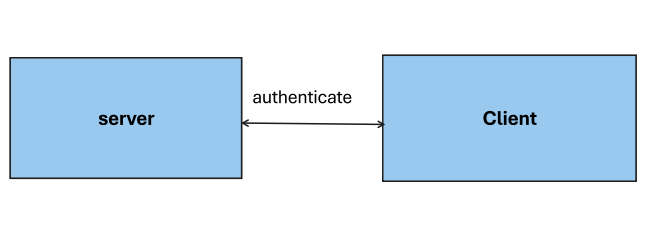
Thumb Rule: "Trust the identity, not the network"

* Core Features:

1. mTLS (Mutual TLS)

Encrypts traffic between services

Both client and server authenticate each other



Prevents:

- Man-in-the-middle attacks
- Unauthorized service access

2. Identity-Based Access (SPIFFE) - Secure Production Identity Framework for Everyone

- Each workload gets a cryptographic identity
- Based on SPIFFE standard

Identity format looks like:

spiffe://cluster.local/ns/default/sa/my-service

Component	Example	Description
Scheme	spiffe://	The standard protocol identifier.
Trust Domain	cluster.local	The boundary of trust (usually your specific K8s cluster).
Namespace	ns/default	The Kubernetes namespace where the pod is running.
Service Account	sa/my-service	The K8s ServiceAccount assigned to that specific workload.

3. Certificate Lifecycle Management

- Handled automatically by istiod:

  - Issues certificates to services
  - Rotates certificates periodically
  - Revokes/validates identities

No manual certificate handling needed

Key Istio CRDs for Security:

- PeerAuthentication → controls mTLS mode (strict/permissive)

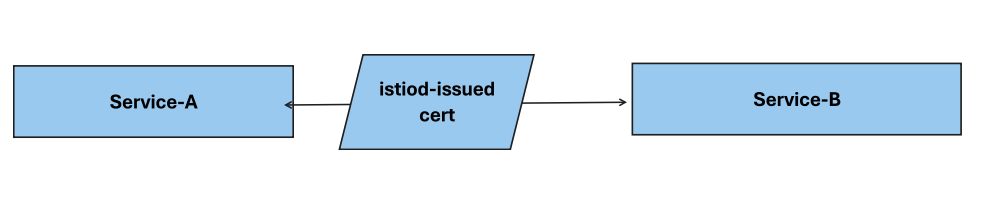
- AuthorizationPolicy → defines who can talk to whom

- RequestAuthentication → handles JWT validation (end-user auth)

Example Flow:

Service A → Service B

1. A presents its certificate (identity)
2. B verifies A via istiod-issued cert
3. Secure mTLS connection established
4. AuthorizationPolicy decides if A is allowed



Outcome:

✓ Every service has a verifiable identity

✓ All communication is encrypted (mTLS)

✓ Access control is identity-driven

✓ Zero reliance on network boundaries

Even inside the cluster, nothing is trusted by default

3. Service Discovery:

Service Discovery allows services inside the mesh to find and communicate with each other dynamically, without hardcoding IPs.

How It Works:

The control plane component istiod continuously watches:

- Kubernetes API Server
- Kubernetes Services and Endpoints

What Istio Knows:

From this data, Istio maintains a real-time registry of:

- Which services exist

- Where they are running (pods/nodes)

- How to reach them (IPs, ports, labels)

This information is pushed to Envoy sidecars for routing decisions.

** Service Naming (DNS-based):

Istio uses Kubernetes DNS naming conventions:

<service-name>.<namespace>.svc.cluster.local

Example:

service-a.default.svc.cluster.local

This resolves to all healthy instances of service-a in the default namespace.

What Happens at Runtime:

Service A → (DNS lookup) → istiod data → Envoy proxy → Service B

- No fixed IPs
- Dynamic load balancing
- Automatically adapts to scaling/failures

WHy it matters: Works seamlessly with auto-scaling, Handles pod restarts without breaking communication
Enables advanced routing (used by VirtualService)

Services don’t need to know where others are — Istio figures it out.

4. Configuration Distribution:

This is how Istio turns your YAML configs into real runtime behavior.

The control plane istiod pushes configuration to all Envoy sidecars dynamically.

* What Gets Distributed?

Istio translates CRDs into Envoy-ready config and sends:

- Routing rules (from VirtualService)

- Traffic policies (from DestinationRule)

- Security policies (mTLS, AuthorizationPolicy)

- Telemetry configs (metrics, logs, traces)

* Protocol Used: xDS APIs

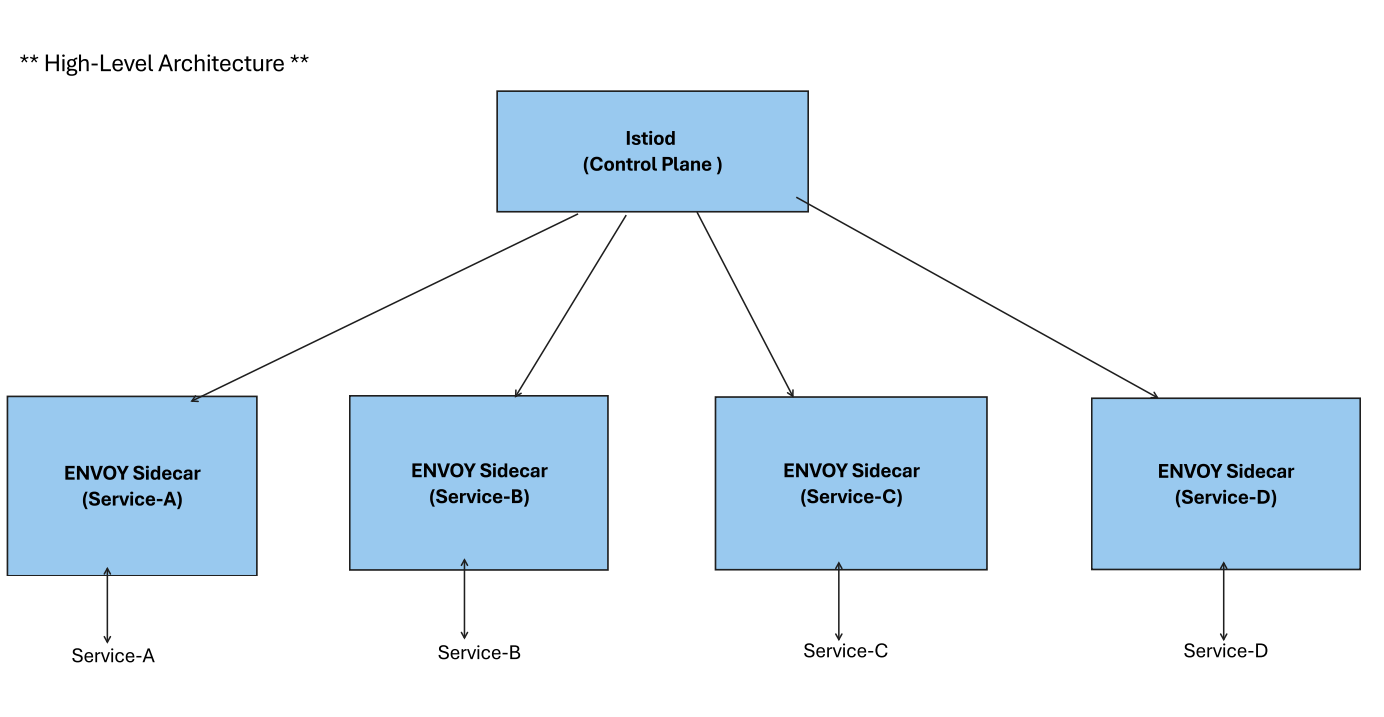
Communication between istiod → Envoy proxies happens via: xDS APIs

Key xDS Types:

- LDS (Listener Discovery Service) → how Envoy listens for traffic
- RDS (Route Discovery Service) → routing rules
- CDS (Cluster Discovery Service) → upstream services
- EDS (Endpoint Discovery Service) → actual service instances

How It Works (Flow):
1. You apply YAML (VirtualService, etc.)
2. istiod reads & processes config
3. Converts it into xDS format
4. Pushes updates to Envoy sidecars
5. Envoy updates instantly (no restart)

** High-Level Architecture **



Feature	**Control Plane (istiod)**	**Data Plane (Envoy)**
Role	Brain	Execution
Handles traffic?	❌ No	✅ Yes
Configures routing	✅ Yes	❌ No
Security policies	✅ Yes	Enforces
Runs where	Central	Sidecar per pod

Real-World Analogy

Concept	**Real Life**
istiod	Airport Control Tower
Envoy Proxy	Airplane Pilot
Services	Passengers
Policies	Flight Rules

* Data Plane & Envoy Proxy:

(Execution Layer of Service Mesh)

-> The Data Plane is where actual traffic flows.

It consists of:

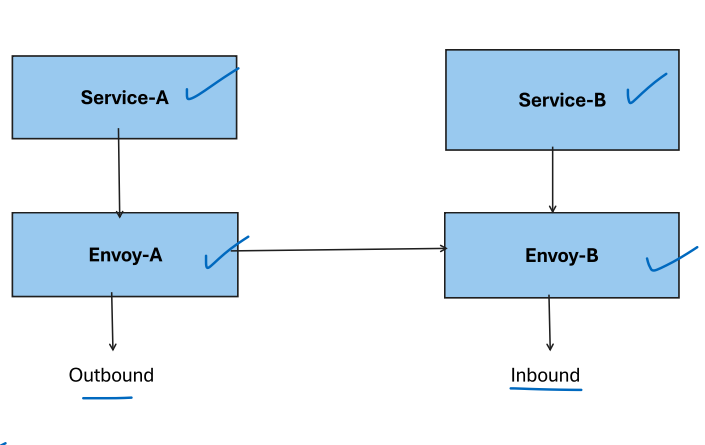
- Envoy sidecar proxies

- Deployed alongside every microservice pod

* Key idea:

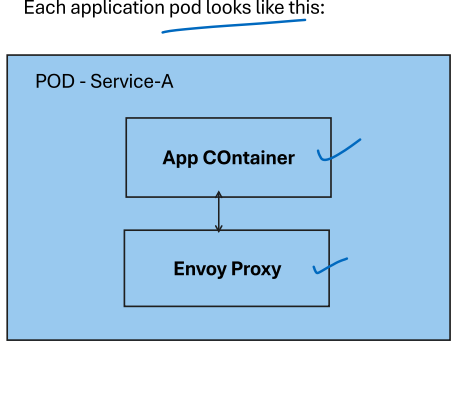
Data plane executes what the control plane (istiod) decides.

* Full Data Plane Flow Diagram:



* Sidecar Proxy Pattern:

Each application pod looks like this:



Envoy sits next to your app, not inside it.

* How Communication Changes :

❌ Without Istio

Service A ──────► Service B

* Direct communication

* App handles:
  - retries
  - timeouts
  - security

Tight coupling ❌

✅ With Istio:

Service A → Envoy → Envoy → Service B

Each service has a sidecar proxy (Envoy) injected.

Outbound traffic → intercepted by local Envoy

Inbound traffic → received via destination Envoy

What Is Envoy Doing?

The proxy (Envoy) transparently handles:

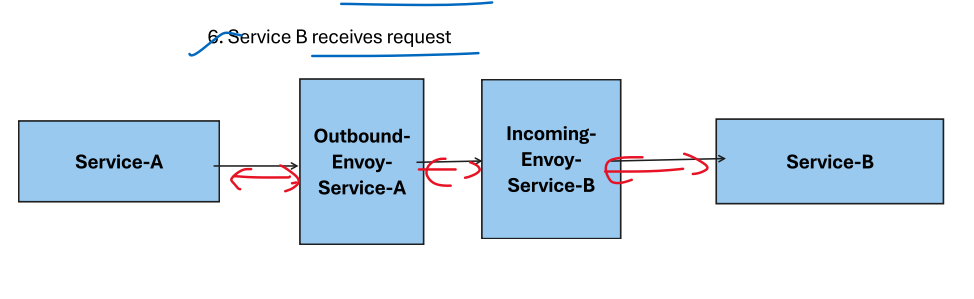
- Traffic routing (VirtualService rules)
- mTLS encryption
- Retries, timeouts, circuit breaking
- Metrics & tracing

* Key Principle:
- Applications don’t talk to each other anymore
They talk to their local Envoy, which then:

- Applies policies from istiod
- Forwards the request to the destination Envoy

Traffic Flow (Step-by-Step)

1. Service A sends request
2. Outbound Envoy intercepts it
3. Applies routing + security rules
4. Sends to Service B’s Envoy
5. Inbound Envoy validates & forwards
6. Service B receives request



* Why This Matters:

- Zero changes to application code
- Consistent policies across all services
- Full observability and control
- Enables all Istio features (traffic, security, etc.)

* Outcome:

✓ No direct service-to-service communication

✓ All traffic is mediated, secured, and controlled

✓ Sidecar proxies act as the data plane

“Apps focus on business logic — Envoy handles everything else.”

* Features:

1. Load Balancing (Handled by Envoy)

Envoy performs client-side load balancing, meaning:

The source service’s Envoy decides where the request goes

No external load balancer required inside the mesh

- Envoy gets service endpoints from istiod

- It maintains a local list of healthy instances

- Chooses a destination using configured policy

* Load Balancing Strategies:
- Round Robin:
  - Requests distributed evenly
  - Simple and widely used

* Least Connections:
- Sends traffic to the least busy instance
- Better for uneven workloads

* Weighted Routing:
- Traffic split based on percentages
- Used heavily for canary deployments

Example (Canary Split)

```
Service A → Envoy
          ├─ 80% → Service B v1
          └─ 20% → Service B v2
```

This is configured using:

- VirtualService → defines 80/20 split

- DestinationRule → defines v1, v2 subsets

* What Envoy Actually Decides:

For every request, Envoy:

- Picks a cluster (service)
- Chooses a subset (version)
- Selects a specific pod (instance)

All in real-time

* Why This Is Powerful
No DNS round-robin limitations
Per-request decision making
Supports:
- retries
- failover
- health checks

* Outcome

✓ Intelligent traffic distribution
✓ Fine-grained control (per request)
✓ Enables safe deployments (canary, blue-green)
“Envoy performs client-side, policy-driven load balancing using real-time endpoint data from istiod.”

* 2. Retry Logic (Handled by Envoy)

Envoy can automatically retry failed requests—completely transparent to your application.

Your app sends one request, Envoy may send multiple attempts behind the scenes.

* What You Can Configure

Using Istio (via VirtualService), you control:

🔁 Retry attempts → how many times to retry
⏱ Per-try timeout → how long to wait per attempt
❗ Retry conditions → when to retry:
- 5xx errors (server failures)
- connection failures
- timeouts

* Example Flow:
Service A → Envoy → Service B

Attempt 1 → fails (500 / 503 etc..)
Attempt 2 → retry
Attempt 3 → success ✅ --> The application never knows retries happened.

* How It Works Internally:

Step1: Envoy detects failure conditions
Step2: Applies retry policy (from config pushed by istiod)
Step3: Re-sends request to:
same instance OR
different healthy instance (better reliability)

* Important Considerations:

Retries are powerful—but must be used carefully:

Too many retries → can overload services
Can increase latency
Should be paired with:
timeouts
circuit breaking

* Why This Matters:

✅ Handles transient failures automatically
✅ Reduces need for retry logic in code
✅ Improves success rate of requests

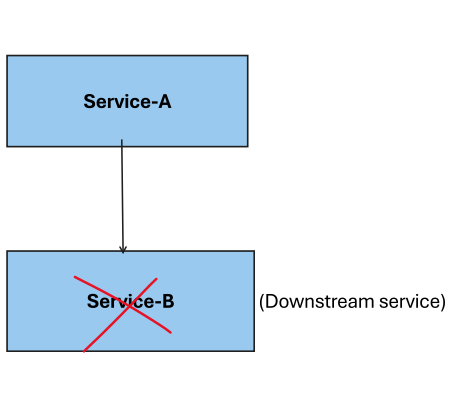
* Outcome:

✓ Higher reliability
✓ Zero code changes
✓ Centralized retry policy

3. Circuit Breaking:

Circuit Breaking prevents cascading failures across services.

If a downstream service (e.g., Service B) starts failing:



- Envoy detects instability

- Stops sending more traffic

- Fails requests immediately (fail fast)

* Why This Is Needed:

Without circuit breaking:

Service B is slow/failing →
Service A keeps retrying →
Threads pile up →
System-wide failure 💥

This is called a "cascading failure"

* What Envoy Controls:

Configured via DestinationRule, Envoy enforces limits like:

- Max connections

- Max pending requests

- Max requests per connection

❌ Outlier detection (eject unhealthy instances)

* Example Scenario:

Service A → Envoy → Service B

Service B starts failing ❌

Envoy:
- Detects high error rate
- Stops sending traffic
- Returns error immediately

Combined with Retries:
Best practice:

Retry → for temporary failures

Circuit breaker → for persistent failures

* Together they create a resilient system

* Outlier Detection (Key Concept)

Envoy can:

Monitor response codes (e.g., 5xx)

Identify “bad” instances

Temporarily eject them from load balancing

-> Only healthy instances receive traffic

* Outcome:

✓ Prevents system overload

✓ Isolates failing services

✓ Maintains overall system stability

* “If a service is clearly struggling, stop hitting it—give it time to recover.”

* “Envoy implements circuit breaking by limiting requests and ejecting unhealthy instances to prevent cascading failures and ensure system stability.”

4. mTLS Encryption:

Envoy enforces mutual TLS (mTLS) for all service-to-service communication.

Every request is:

- Encrypted in transit
- Authenticated on both sides

* What Envoy Actually Does:
For every request between services:

Service A → Envoy → Envoy → Service B

* Outbound Envoy:
Attaches client certificate (identity)
Initiates secure TLS connection

* Inbound Envoy:
Verifies client certificate
Confirms identity is valid

* Secure channel established ✅

✔ What’s Automatic

All of this happens without any application code changes:
✅ Certificate issuance
✅ Certificate rotation
✅ Trust validation

Handled by istiod - “Every request carries a verified identity and travels through an encrypted tunnel—by default.”

5. Metrics & Observability:

Envoy automatically collects rich telemetry for every service-to-service request.

No code changes required—the sidecar does it all.

What Envoy Generates:

* Key metrics include:

- Request count – number of requests per service

- Latency – e.g., P50, P95, P99 response times

- Error rate – % of failed requests (5xx, 4xx)

Metric Type	**Example**
RPS	120 requests/sec
Latency	P99 = 350ms
Errors	0.5% failed requests

Integration:

Envoy pushes metrics to:

Prometheus – collects & stores metrics
Grafana – visualization & dashboards

* How It Works:
- Envoy intercepts request/response
- Measures latency, status codes, request size
- Emits metrics to Prometheus via Istio’s telemetry pipeline
- Dashboards show real-time performance

* Why This Matters:
✅ No code instrumentation needed
✅ Real-time service health visibility
✅ Enables SLOs, SLIs, and alerts
[SLI (Service Level Indicator) , SLO (Service Level Objective)]
✅ Supports performance tuning & root cause analysis

* 6. Logging & Tracing:

Envoy provides transparent logging and tracing for every request flowing through the mesh.

Distributed Tracing:

Tools supported:

- Jaeger
- Zipkin

* Trace example:

User → API Gateway → Service A → Service B → Database

Each hop is recorded automatically, giving end-to-end visibility.

Inbound vs Outbound Traffic:

Type	Description
Outbound	App → Envoy → Network
Inbound	Network → Envoy → App

* What Envoy Handles:

- Tracing headers (e.g., x-request-id, traceparent)

- Logging per request & per route

- Context propagation across services

No application changes needed—traces and logs are captured automatically.

Envoy intercepts both directions, enabling:

- Tracing headers propagation
- Metrics collection
- mTLS encryption
- Routing & retries

Why This Matters:

✅ End-to-end visibility for debugging
✅ Quickly locate bottlenecks or failures
✅ Understand request flows for complex microservices

Outcome:

Full observability from user request → backend

Both inbound and outbound traffic are controlled and traced

Enables root cause analysis and performance optimization

** Istio & helm installation is done:

** Demo: Taking sample application "BOOKING info" App

STEP 1: Enable Sidecar Injection

GCP - GKE - Certifications:

1. Google Professional Cloud DevOps Engineer

CI/CD (Day 8)
Monitoring & Logging (Day 5)
GKE (Day 4, 9)
Service Mesh (Day 13–16)
Reliability + scaling

2. Google Professional Cloud Architect

GCP architecture (Day 1)
Networking (Day 11–12)
Migration strategy (Day 6)
Security & IAM (Day 9, 11)
Anthos (Day 13–14)